STARTER

Simple distribuTed smART conTrollER

Christian Brunello

Copyright © 2026 Christian Brunello.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Table of Contents

Introduction 2
The Search for Intelligence 2
The Pivot: Programmable Intent 2
Beyond Heating: The Monero Bonus 3
Giving Back to the Community 3

1 STARTER Concepts 4
1.1 Unlimited Scalability 4
1.2 Redefining the Foundations: Inputs, Outputs, and Events 4
1.2.1 Input: More than just Sensors 5
1.2.2 Output: A Channel for Action 5
1.2.3 Events: The Language of Change 5
1.2.4 Logic: Applying Our Intelligence 5
1.2.5 Analysis: Data-Driven Decisions 5

2 Using STARTER: From Theory to Practice . . . 7
2.1 A Quick Disclaimer 7
2.2 System Preparation 7
2.2.1 Dependencies 7
2.2.2 Database Initialization 7
2.2.3 Download and Installation 7
2.2.4 Configuration 8
2.2.5 Where to go from here? 8
2.3 The Rules File: Where the Magic Happens 8
2.3.1 Identifiers: The Power of Abstraction 9
2.3.2 The Anatomy of a Logic Block 9
2.3.3 Connecting the Dots: Events in Action 9
2.3.4 Dynamic Control: The TRIGGER Construct 10
2.3.4.1 Choosing Your Tool 10
2.3.4.2 Examples 10
2.3.5 Variables: The Connective Tissue 11
2.3.5.1 The Power of Propagation 11
2.3.5.2 How the System "Thinks" 11
2.3.5.3 Why Use Variables? 11
2.3.6 Expressions: The Engine of STARTER 11
2.3.6.1 Constant Values and Types 12
2.3.6.2 Boolean Logic: Everything is a Number 12
2.3.6.3 Comparison and Logical Operators 12
2.3.6.4 Mathematical Operators 12
2.3.6.5 Built-in Functions 13
2.3.6.6 I/O Flags in Rules 13
2.3.7 The Mechanics of Evaluation: Snapshots vs. Live Streams . . 13

2.3.7.1 The EVAL Function: Killing the Link 14
2.3.7.2 Why This Matters: Architectural Elegance 14
2.3.7.3 The Boot-Time Paradox 14
2.3.8 Preprocessing: The Secret Weapon 15
2.3.8.1 Power Unleashed: The M4 Default 15
2.3.8.2 Swapping the Engine 15
2.3.9 Command Line Options: Taking the Reins 15
2.3.9.1 Advanced Orchestration: Segmenting the System 16
2.3.9.2 The Override Hierarchy 16

3 System Architecture: Under the Hood 17
3.1 A Truly Distributed Ecosystem 17
3.1.1 ZeroConf: Plug & Play Reality 17
3.1.2 Communication via D-Bus RPC 17
3.1.3 The Operational Loop 18
3.2 The Debug Server 18
3.2.1 Connecting to the Debug Endpoint 18
3.2.2 Debug Methods 18
3.2.3 Debug Signals 19
3.2.4 Pause Semantics 20

4 The starter-core Library: Building Your Own Ecosystem 21
4.1 The Skeleton: `first-service.c` 21
4.1.1 Compiling and Running 22
4.1.2 Breaking Down the Code 22
4.2 Adding Substance: Inputs and Outputs 22
4.2.1 The Anatomy of an I/O Object 23
4.2.2 The Power of Flags: Customizing I/O Behavior 23
4.3 Signals: The Reactive Core 24
4.3.1 Input vs. Output: What's the difference? 24
4.4 The Grand Finale: Bridging Code and Logic 24
4.4.1 The Living Service: `first-service-v3.c` 24
4.4.2 The Rule: Closing the Loop 26
4.4.3 The Result: A Symphony of Data 26

5 Conclusions: The World is Your Input 27
5.1 Thinking in STARTER 27
5.1.1 The Power of Generalization 27
5.1.2 Where Will You Take It? 27

Appendix A Database Schema & Initialization . . 28

Appendix B API Reference 30
B.1 Error Handling: The `STError` Domain 30
B.1.1 STError Enumeration 30
B.1.2 Error Quarks and Macros 30
B.2 API Definitions: Constants and Enumerations 31
B.2.1 I/O Flags 31
B.2.2 STUnit Enumeration 31
B.2.3 Special Constants 32
B.3 STInput: The Sensor Abstraction 32
B.3.1 Lifecycle Management 32
B.3.2 Metadata Accessors 33
B.3.3 Range and Value Control 33
B.4 STOutput: The Actuator Abstraction 34
B.4.1 Lifecycle Management 34
B.4.2 Metadata Accessors 34
B.4.3 Value and State Control 34
B.5 Network Discovery: mDNS and Service Location 35
B.5.1 STMDNSService: Service Representation 35
B.5.1.1 Property Management 36
B.5.2 STMDNS: The Discovery Engine 36
B.5.2.1 Signals 36
B.6 System Statistics: Performance Monitoring 37
B.6.1 STStats: The Telemetry Container 37
B.6.1.1 Metric Accessors 37
B.6.2 STStatsCollector: The Abstract Engine 37
B.6.3 STProcessStatsCollector: Process Monitoring 37
B.7 Services and Communication: Server and Client 38
B.7.1 STServer: Publishing a Service 38
B.7.1.1 Service Metadata 38
B.7.2 STClient: Consuming a Service 38
B.7.2.1 Client Connection State 39
B.7.2.2 Starting a Client 39
B.7.2.3 Remote Resource Access 40

Appendix C GNU Free Documentation License .. 41

Introduction

This project was born out of a personal necessity: I needed to stop bleeding money on home heating.

At first, I tried the standard route–using a smartphone app to schedule the boiler. But I quickly learned that savings were inversely proportional to comfort. My partner and I would celebrate a lower gas bill while shivering in extra sweaters.

With a radiator system and a hybrid boiler, we ran the gamut of options: hourly scheduling, integrated weather compensation curves, and continuous low-power operation. Nothing worked. It was a binary choice between comfort or savings; there was simply no middle ground.

The Search for Intelligence

I looked for off-the-shelf solutions to make the boiler "smarter," but I hit a dead end. While there were plugins available to *control* the hardware, the actual "intelligence" was missing. I thought: "*We live in the age of AI–why can't we apply that to home automation?*"

That's when the project truly began. My initial plan was to use a "Lite" version of TensorFlow. Since I'm not wealthy and couldn't afford powerful servers, I invested my expertise instead of my cash. I envisioned a core that could toggle between "learning mode" and "active mode." It was an exciting prospect, but the AI path proved rocky. It required massive amounts of training data, and the results were unpredictable. I couldn't risk spending the rest of the winter in the cold just for a neural network experiment.

The Pivot: Programmable Intent

I decided to strip the problem down to its core objective: *Giving the system clear instructions on how to behave based on the environment.*

To give instructions, you need to communicate. To communicate, you need a *channel* and a *protocol*. I realized the channel could be a simple file, and the protocol a straightforward, human-readable language. This was the breakthrough that finally hit the target: *halving energy consumption while actually increasing comfort.*

I achieved this through two fundamental logic rules:

Efficiency Switching
: Force the system to gas-only mode the moment environmental conditions make the heat pump inefficient.

Dynamic Flow Control
: Set the boiler's flow temperature based on:

  - A weighted average of the forecasted daily temperature swings.
  - A calculated offset based on a physical constant I discovered for my home: at 0°C outdoors, a 48°C flow temperature perfectly balances the house's heat loss.

Beyond Heating: The Monero Bonus

Once the core was stable, more automations followed. The most satisfying one? Managing my photovoltaic surplus. When my solar panels are overproducing and the grid voltage spikes near 250V, the system automatically triggers my mining rigs to mine Monero. It's the ultimate way to turn excess voltage into digital value.

Giving Back to the Community

I believe this system has applications far beyond my own living room. I'm sharing it so others can benefit from these optimizations just as I have.

I am a firm believer in *Free Software* (in the true Stallman sense), and this is my contribution to the community. To ensure it remains free and open for everyone to improve, I am distributing the Core under the GPL license and the Library under the LGPL.

1 STARTER Concepts

Now that we've covered the "why," I want to dive into the "how". Let's start with the name. STARTER is an acronym for *Simple distribuTed smART conTrollER*, and every letter reflects a core design principle.

Simple

- Adding a new module is as easy as powering it on.
- You don't need to be a senior dev to manage it; the control logic uses a near-natural language that anyone can understand.

distribuTed

- Every component operates over the network. It's designed to be lightweight, running smoothly even on low-spec, legacy hardware.

smART

- Services connect and disconnect on the fly, with rules applied automatically in real-time.
- It doesn't just follow a schedule; it reacts to environmental changes.
- It allows us to translate our specific will into action using a near-human syntax.

conTrollER

- Through our custom rules, it acts as the "brain," driving every peripheral exactly as instructed.

I chose this name because I wanted it to be clear that this core is just the *starting point*. Any user can download it, plug in their own services, and begin coordinating them. Currently, my setup runs on a single computer managing a mix of services. However, the underlying architecture is generic: it can be applied virtually anywhere you need to define the logic: "*When these conditions are met, react this way*".

1.1 Unlimited Scalability

Because the system is modular and network-based, it is *completely scalable*. You can start with a basic home automation setup and grow into a massive distributed network with multiple cores, each dedicated to a specific task. We aren't just talking about managing a house; we're talking about scaling to a warehouse, a farm, a neighborhood, or even an entire city.

Before we get into the technical weeds, I want to take a moment to look at the foundational concepts of automation. Through this manual, my goal is to help you see automation through an *abstract lens*. Once you grasp the abstraction, you'll realize how STARTER can be adapted to solve almost any challenge you throw at it.

1.2 Redefining the Foundations: Inputs, Outputs, and Events

To understand how STARTER works, we need to strip away our preconceptions of what automation looks like and embrace a more abstract perspective.

1.2.1 Input: More than just Sensors

When I first sat down to design STARTER, I asked myself: *"What exactly is an input?"* The instinctive answer is "a sensor", but that's far too limiting. In this system, an *Input* is any incoming data point, whether it originates from physical hardware or an abstract digital service.

It could be a humidity sensor in your basement, but it could just as easily be a weather API, the current exchange rate of the Euro, or a real-time cryptocurrency price. If a service can feed data into the system, STARTER treats it as an input.

1.2.2 Output: A Channel for Action

We usually think of an *Output* as something physical that clicks or turns–like a relay switching on a light. But at its core, an output is simply *any "channel" where data can be written.*

By treating outputs as abstract "writeable" points, we can control almost anything. For example, I created a small "Explain" service that gives my home automation a voice; it literally tells me when something changes. If you can send a value to it, STARTER considers it an output.

1.2.3 Events: The Language of Change

In traditional automation, "events" are often binary: a gate opened, a relay closed. But let's broaden our perspective. Since inputs and outputs are abstract, an *Event* is simply a *state change.*

An event could be "it's getting dark," "it's raining," or "100 cars have passed the gate." It could be "my EV battery is charged" or "Bitcoin just hit €60,000." *Open your mind*: if the data changes, it's an event you can act upon.

1.2.4 Logic: Applying Our Intelligence

In automation, logic is everything. Usually, this means "if X happens, then do Y." In 90% of existing systems, these rules are hard-coded—buried inside the software where they are difficult to change.

In STARTER, the logic is *external.* It is defined entirely by the user through custom rules. This shifts the power back to you: we aren't limited by how "smart" the software is; instead, we have a system that *applies our own intelligence.*

Take my heating setup: when the outdoor temperature shifts, STARTER recalculates the boiler's flow temperature. This, in turn, triggers a re-evaluation of the heat pump's efficiency based on ambient humidity. The system doesn't "understand" thermodynamics (and honestly, neither do I), but it follows the rules I've written. If the inputs match a certain pattern, the outputs react accordingly.

> **The takeaway**: If you can encode a condition into an input or an output, you can master it.

1.2.5 Analysis: Data-Driven Decisions

Perfecting my boiler's weather compensation curve wasn't an overnight success. It took weeks of measuring and tweaking to find my "sweet spot": a 48°C flow temperature to perfectly balance heat loss when it's 0°C outside.

To get there, I couldn't rely on guesswork like "*maybe if I turn it up a bit...*". I needed cold, hard facts.

That's why I built STARTER to be *data-driven* from the ground up. Every single event—every change in input or output—is automatically logged in a database. The best part? The database belongs to you; STARTER simply manages the flow of information.

Having this historical data is a game-changer. I personally use `Grafana` to visualize everything:

- A dashboard comparing home consumption vs. grid demand.
- Real-time power usage and voltage fluctuations.
- Logs showing exactly when my mining rigs kick in.

Once again, I want you to think abstractly: we have the data we've managed to encode. *What that data represents is entirely up to you*!

2 Using STARTER: From Theory to Practice

Now that we've explored the philosophy of automation and (hopefully) expanded our perspective on what's possible, it's time to get our hands dirty. Let's look at how to download and install STARTER on your system.

2.1 A Quick Disclaimer

Before we dive in, it's important to remember that STARTER is, first and foremost, a "Core". While the included examples are great for practice and getting a feel for the logic, "the real deal" happens when you start plugging in your own services and defining your unique rules. Think of the Core as the conductor and the services as the orchestra; you'll need to bring your own instruments to the stage.

2.2 System Preparation

STARTER is designed to run on a full GNU/Linux environment.

For the sake of efficiency (and, let's be honest, a bit of developer "laziness"), I developed the system on a *Debian-based* distribution. Because of this, the instructions below are tailored for that environment. If you are using a different distro, the core concepts remain the same, but you might need to adapt some specific commands or package names to fit your setup.

2.2.1 Dependencies

To compile STARTER, you'll need to prepare your environment. The system relies on several core libraries for networking and data management.

Required packages:

- `glib-2.0` and `gio-2.0` (version 2.44 or later; the backbone for system abstractions)
- `avahi-client` and `avahi-glib` (For zero-configuration networking/service discovery)
- `libmysqlclient-dev` (To enable database connectivity)

Optional (but recommended):

- `flex`, `bison`, and `makeinfo` (Useful for specific build tasks or documentation)

2.2.2 Database Initialization

Before launching the Core, you need a place for it to store its "memory." You must create a MariaDB/MySQL database—either locally or on a remote server—named `STARTER`.

The database requires three specific tables: `labels`, `history`, and `stats`. You can find the exact SQL schema and creation scripts in Appendix A [Database Schema & Initialization], page 28.

2.2.3 Download and Installation

STARTER follows the standard GNU packaging conventions and utilizes `autotools`. If you've ever compiled software on Linux, you'll feel right at home. Run the following commands from your terminal:

```
cd starter-directory
./configure
make
sudo make install
```

To build the optional remote debug server (see Section 3.2 [The Debug Server], page 18), pass `--enable-debug-server` to `configure`:

```
./configure --enable-debug-server
```

The Core will be installed (typically under `/usr/local`) and automatically configured as a `systemd` service. Note that the service is not enabled for auto-boot by default—we'll leave that decision to you once everything is configured.

2.2.4 Configuration

The final step is to point STARTER to your database. Instead of messy config files, STARTER uses `gsettings` for a clean, integrated configuration. Use the command line to set your credentials:

```
gsettings set org.starter.core db-host 'your_ip_or_hostname'
gsettings set org.starter.core db-port port
gsettings set org.starter.core db-user 'your_username'
gsettings set org.starter.core db-pass 'your_password'
```

Note: Ensure the database user has full Read/Write permissions.

2.2.5 Where to go from here?

Your Core is now standing by, ready for orders. What happens next depends entirely on your objective—whether it's home automation, industrial control, or building a smart city.

1. **Install your services**: Deploy the specific modules you need for your project.
2. **Fire them up**: Once the services are live, they will automatically announce themselves to the Core.
3. **Define the Logic**: Write your rules and watch the system transform from a collection of parts into an intelligent, autonomous environment.

2.3 The Rules File: Where the Magic Happens

The rules file is the true heart of your system. This is where your abstract ideas turn into reality: "*If it rains, close the shutters*", or "*If it's a sunny day, run the irrigation at sunset*".

The live rules file is typically *installdir*`/etc/starter/rules.conf` (create it on the target host; package installs only `rules.conf.example` so upgrades never overwrite your production rules). To help you master the STARTER syntax, we'll use the provided example services. Let's open the file and write our very first logic block:

```
beep:status {
    SET TRUE WHEN time:hour == 7 && time:minute == 0;
    SET FALSE WHEN TRUE;
}
```

If you now launch `starter-core`, `time-example`, and `beep-example`, you've just built an alarm clock that rings every morning at 7:00 for exactly one minute. Simple, right? But let's look under the hood to see how it actually works.

2.3.1 Identifiers: The Power of Abstraction

In the code above, we see terms like `beep:status`, `time:hour`, and `time:minute`. In STARTER language, these are *Identifiers*.

Remember our mantra: *generalize*. Let's break down what these identifiers represent in this specific context:

`beep:status`
: This is an *Output* provided by the *ST:Beep* service. When set to `TRUE`, it triggers your computer to beep twice per second; when `FALSE`, it stays silent.

`time:hour`
`time:minute`
: These are *Inputs* exposed by the *ST:Time* service. This service tracks the current system time and broadcasts it as three distinct inputs: *hours*, *minutes*, and *seconds*.

By writing those two rules, you've told the system: "*At 7:00 AM, start beeping. For the rest of the time, stay quiet*".

2.3.2 The Anatomy of a Logic Block

Every logic block in STARTER follows a consistent, clean format:

```
target {
    rule; ...
}
```

target Represents *WHAT* you want to control (the Output).

rule Represents *HOW* and *WHEN* it should react.

If we were to translate our "alarm clock" block into plain English, it would look like this:

```
BEEP {
  Set to Active when it is exactly Seven o'clock;
  Set to Off whenever the first condition isn't met (Default to False).
}
```

2.3.3 Connecting the Dots: Events in Action

In the previous section, we defined *Events*, *Inputs*, and *Outputs*. Let's see how they dance together in our alarm clock example:

The Event: "The current time has changed."
The Inputs: time:hour and time:minute.
The Output: The computer's beep (On or Off).

For STARTER, the logic is straightforward: every time one of the inputs (*time:hour* or *time:minute*) changes, the entire block is *re-evaluated*. The system checks the rules from top to bottom and sets the target (*beep:status*) based on the *first rule that evaluates to* `TRUE`.

Simple, right? But now, let's go back to our mantra: *Abstraction.*

What else could *beep:status* represent? And what other data could we feed in as an input? Let's push the boundaries. Open your rules file again and swap the code for this:

```
beep:status {
    SET TRUE WHEN oil:price < 100;
    SET FALSE WHEN TRUE;
}
```

Now, launch `starter-core`, `beep-example`, and `oil-example`. If you suddenly hear your computer beeping, it means the price of oil has just dropped below 100 a barrel!

> **Think about the implications:** What if, instead of a simple beep, that output was a service that sent a push notification to your phone? Or a service that executed a buy order on an exchange? In STARTER, there is no difference between a temperature sensor and a stock market index—*data is data.*

2.3.4 Dynamic Control: The TRIGGER Construct

Now that you've got a feel for how to define a target, I want to show you the second main construct of the STARTER language:

```
TRIGGER WHEN condition SET target = value;
```

While the first method (the `target { ... }` block) defines a set of rules for a *single objective*, the `TRIGGER` allows you to define *multiple assignments linked to a specific event.*

Let's go back to our oil price example:

```
TRIGGER WHEN oil:price < 100 SET beep:status = TRUE;
TRIGGER WHEN oil:price > 100 SET beep:status = FALSE;
```

We've achieved the exact same result as before, but with a different syntax. This leads to a natural question: *When should you use* `TRIGGER` *instead of a target block?*

2.3.4.1 Choosing Your Tool

The two constructs are designed for different architectural needs:

The Target Block (`target { ... }`*):*
: Use this when you need to drive *one specific output* based on various complex conditions. It's perfect for centralized control over a single device (like a lightbulb that has many reasons to be on or off).

The `TRIGGER`*:*
: Use this when you want to *synchronize multiple actions* to a single event. It's the "domino effect" of automation.

2.3.4.2 Examples

Scenario A: "Turn the light on if it's dark, off if it's bright."
: Here, you are controlling *one* light based on *one* sensor. We use the *Target Block* with multiple rules inside it.

Scenario B: "When I turn on the first lamp, automatically turn on the second and third lamps too."
: Here, you are triggering a chain reaction across *multiple* outputs. We use a `TRIGGER` to assign the value of the first lamp to all the others simultaneously.

2.3.5 Variables: The Connective Tissue

Until now, we've used "Target" to refer to a physical or abstract Output. But STARTER allows for a powerful third option: *Variables*.

In STARTER, a variable can act as both a *Target* (something you write to) and an *Input* (something you read from). This allows you to create intermediate logic that simplifies even the most complex scenarios.

2.3.5.1 The Power of Propagation

Let's look at how variables can streamline your code:

```
VAR turn_on_lamps = 0;

turn_on_lamps {
    SET 1 WHEN time:hour >= 22 || time:hour <= 5;
    SET 0 WHEN TRUE;
}

TRIGGER WHEN turn_on_lamps == 1 SET Lamp1 = 1, Lamp2 = 1, Lamp3 = 1;
TRIGGER WHEN turn_on_lamps == 0 SET Lamp1 = 0, Lamp2 = 0, Lamp3 = 0;
```

2.3.5.2 How the System "Thinks"

What's happening under the hood? It's a chain reaction of events:

1. *The Triggering Event*: The clock ticks. *time:hour* changes.
2. *First Evaluation*: STARTER sees that *turn_on_lamps* depends on the time. It recalculates the variable.
3. *Event Propagation*: If the value of *turn_on_lamps* changes (e.g., from 0 to 1), that change *is itself a new event.*
4. *Second Evaluation*: Since our `TRIGGER` rules are "listening" to *turn_on_lamps*, they fire immediately, updating all your lamps at once.

2.3.5.3 Why Use Variables?

As you can see, if you have ten different lamps, it's much more efficient to define one variable and a couple of triggers than to write ten identical, complex logic blocks.

But let's go back to our mantra: *Abstraction!*

A variable doesn't just have to be a "switch" for lights. It could represent a "System State" (e.g., *Home, Away, Night, Party*). By changing one variable, you can trigger a cascade of hundreds of outputs across your entire network. You aren't just writing rules anymore; you are designing a *behavioral ecosystem.*

2.3.6 Expressions: The Engine of STARTER

Up until now, we've used simple expressions for basic evaluations. In reality, *every condition or assignment* in STARTER is an expression that can range from a single number to a complex mathematical formula.

2.3.6.1 Constant Values and Types

All numerical constants in STARTER are handled as *double-precision floating-point numbers.* Whether you're comparing *time:hour* to `7` or calculating a precise voltage offset, the system provides high precision. While specific limits depend on your hardware, even on a 32-bit machine, the margin for error is negligible for automation purposes.

2.3.6.2 Boolean Logic: Everything is a Number

In STARTER, `TRUE` and `FALSE` are simply aliases for constants:

`FALSE` Is exactly `0`.

`TRUE` Is exactly `1`.

However, the system evaluates *any non-zero value as* `TRUE`. This is a classic programmer's concept, but it's vital for clever automation. For example:

```
beep:status {
    SET TRUE WHEN time:minute;
    SET FALSE WHEN TRUE;
}
```

Why would this make the computer beep almost all the time? Because *time:minute* is only `0` at the very start of the hour. For every other minute (1 through 59), the value is non-zero, which the system interprets as `TRUE`. The beep only stops for that first minute of the hour!

2.3.6.3 Comparison and Logical Operators

You can relate different expressions using standard operators:

Operator	Description
`==`, `!=`	Equal to / Not equal to
`<`, `<=`	Less than / Less than or equal to
`>`, `>=`	Greater than / Greater than or equal to
`&&`	*AND*: True only if both expressions are true
`||`	*OR*: True if at least one expression is true

You can use *parentheses* `()` to group sub-expressions and control the evaluation order. Want an alarm that rings at 7:00 AM *OR* whenever oil prices drop?

```
SET TRUE WHEN (time:hour == 7 && time:minute == 0) || oil:price < 100;
```

2.3.6.4 Mathematical Operators

STARTER isn't just for logic; it's for math. It supports `+`, `-`, `*`, `/`, and the unary minus `-`. Standard mathematical precedence applies (multiplication/division before addition/subtraction), but you can use parentheses to force your own order.

```
2 * 1 + 4 = 6
2 * (1 + 4) = 10
time:minute + (time:hour * 60) = Total minutes since midnight.
```

2.3.6.5 Built-in Functions

To simplify complex rules, STARTER provides a set of powerful functions:

`MIN(arg1, arg2, ...)`
: Returns the lowest value in a list.

`MAX(arg1, arg2, ...)`
: Returns the highest value in a list.

`ROUND(value)`
: Rounds to the nearest whole integer.

`LIMIT(value, min, max)`
: Clamps a value. If it goes below *min*, it returns *min*; if above *max*, it returns *max*.

`CHANGED(id)`
: This is a special one. It returns `1` (`TRUE`) only if the value of that specific *id* has changed since the last evaluation. It's perfect for triggering actions only on a fresh update.

2.3.6.6 I/O Flags in Rules

Every Input and Output carries a `flags` field (see Section B.2.1 [I/O Flags], page 31). In `rules.conf` you can read or write those bits with the `FLAG` qualifier on the same targets and expressions you already use—no new statement kinds.

Read (any defined flag name) — yields `1` or `0`:

```
SET Manual:OutWest WHEN Manual:OutWest FLAG USR1;
SET 0 WHEN Lamp:switch FLAG MANUAL_OVERRIDE;
```

Write — only `USR1`{dots}`USR4` actually change state. The syntax accepts system flags too, but the engine logs a warning and skips the write:

```
Manual:OutWest FLAG USR1 {
    SET FALSE WHEN TIME:hour == 0;
}

TRIGGER WHEN TIME:hour == 0
  SET Manual:OutWest FLAG USR1 = FALSE;
```

Writing a USR bit uses the peer `SetOutput` / `SetInput` methods (value unchanged, flags updated); `InputChanged` / `OutputChanged` already carry the new flags to the Core. Variables do not have flags; they may only *reference* a live `id FLAG USRn` expression.

2.3.7 The Mechanics of Evaluation: Snapshots vs. Live Streams

Understanding how STARTER "thinks" when it encounters an expression is the difference between a static setup and a truly fluid system. Let's break down a simple logic gate:

```
time:second == 0;
```

To evaluate this, the system doesn't just look at a stored number; it polls the value of *time:second* at the exact millisecond the calculation is triggered. If it catches the clock at the top of the minute, the result is `TRUE`. Otherwise, it's `FALSE`.

But here's where it gets interesting: *Variables*.

```
VAR second;
second = time:second;
```

If you're coming from traditional programming, you might expect *second* to hold a static "snapshot" of the time when that line was executed. *In STARTER, it doesn't*. By default, a variable acts as a *Live Reference*. Using *second* or *time:second* in your rules is functionally the same—it's a dynamic link that updates in real-time as the clock ticks.

2.3.7.1 The EVAL Function: Killing the Link

Sometimes, you don't want a live stream; you want to "freeze" a moment in time. This is where the `EVAL()` function comes into play. It forces the system to calculate the final result of an expression and store it as a static constant.

The "Cheat Sheet":

`value = 1;`
: Hard-coded Constant.

`value = time:hour;`
: *Dynamic Reference* (The value evolves with the hour).

`value = EVAL(time:hour);`
: *Frozen Value* (Locks the hour at the moment of assignment).

`value = 1 + time:hour;`
: *Dynamic Calculation* (The result updates as time moves).

`value = EVAL(1 + time:hour);`
: *Static Result* (Calculates the sum once and stores the total).

2.3.7.2 Why This Matters: Architectural Elegance

This "Reference-first" approach allows you to build a sophisticated, human-readable logic tree without repeating yourself:

```
VAR its_seven_o_clock = (time:hour == 7 && time:minute == 0);

beep:status {
    SET TRUE WHEN its_seven_o_clock;
    SET FALSE WHEN TRUE;
}
```

By using a reference (*its_seven_o_clock*), you've created an *Alias*. Every time STARTER checks the *beep:status*, it follows that alias back to the original time logic. It's clean, it's abstract, and it's incredibly powerful.

2.3.7.3 The Boot-Time Paradox

A quick "gotcha" for the pros:

```
VAR startup_hour = EVAL(time:hour);
```

You might intend to capture the exact hour the system starts. However, at the microsecond of boot-up, the *ST:Time* service might still be authenticating on the network. If the input doesn't exist yet, `EVAL` can't do its job. Don't worry—STARTER handles this gracefully by

performing the evaluation as soon as the required service makes its first appearance on the grid.

2.3.8 Preprocessing: The Secret Weapon

The final layer of the STARTER language is *Preprocessing*. Before the Core ever reads your rules, the file is passed through an external program that modifies the text on the fly. What STARTER actually executes is the *output* of that program.

If you've spent time in the trenches with C or C++, this concept will feel familiar—though STARTER's implementation is even more flexible. If you're not a programmer, think of it this way: the preprocessor is like an automated editor that prepares your "manuscript" before the publisher (the Core) sees it.

2.3.8.1 Power Unleashed: The M4 Default

By default, STARTER uses `m4`, the powerhouse preprocessor of the GNU system. It is incredibly deep—too deep for a single manual—but it allows for advanced maneuvers like macros, file inclusions, and conditional text generation.

While I won't turn you into an `m4` expert here, you need to know it's there because it gives you the ability to manage massive, complex sets of rules without repeating yourself. If you're curious, the official `m4` manual is a rabbit hole worth exploring.

2.3.8.2 Swapping the Engine

The beauty of STARTER's philosophy is that you aren't locked in. Don't like `m4`? You can swap the preprocessor for any command-line tool you prefer.

Returning to our `gsettings` configuration, you can change the preprocessor with a single command. For example, if you want to bypass preprocessing entirely and just read the file "as-is," you can use the standard `cat` command:

```
gsettings set org.starter.core rules-preprocessor 'cat {F}'
```

In this syntax, `{F}` is a placeholder that STARTER automatically replaces with the path to your rules file. This flexibility means you could even use a Python script, a custom parser, or a simple text filter as your preprocessor.

2.3.9 Command Line Options: Taking the Reins

While STARTER is designed to run quietly in the background as a system service, there are times—especially during debugging or initial setup—when you'll want to launch it manually.

When you run STARTER from the terminal, you can pass several options to override the default system settings. This is incredibly useful for testing a new set of rules or increasing the log detail without touching your permanent configuration.

The supported options are:

Option	Long option	Description
`-h`	`--help`	Show all available help options and exit.
`-n`	`--no-discovery`	Disable the automatic discovery of network devices/services.
`-m`	`--service-match`	Regular expression to filter recognized services. Default: `'^(STARTER:|ST:).*'`

`-R`	`--rules`	Specify a custom path to a rules file (overrides the default `rules.conf`).
`-P`	`--rules-preprocessor`	Define a specific command to preprocess the rules file on the fly.
`-V`	`--verbosity`	Adjust the detail of the output logs (essential for troubleshooting).
`-v`	`--version`	Print the current version of the Core and exit.
	`--debug-server-bind`	Listen address for the remote debug server (`HOST:PORT`). Available only when built with `--enable-debug-server`. Default: `0.0.0.0:17373`. See Section 3.2 [The Debug Server], page 18.

2.3.9.1 Advanced Orchestration: Segmenting the System

The new `--service-match` (or `-m`) option is a powerful tool for advanced users who wish to split their automation into multiple independent domains.

By default, the Core listens to any service starting with `STARTER:` or `ST:`. However, by modifying this regular expression, you can run **multiple Cores** on the same network without interference. Each Core will only "see" and manage services whose names match its specific pattern.

For example, you could have:

- A Core managing only critical infrastructure: `starter-core --service-match '^ST:CRITICAL:.*'`
- A Core managing leisure or lab services: `starter-core --service-match '^ST:LAB:.*'`

This segmentation ensures that a failure or a configuration error in one domain (like the lab) does not affect the stability of another (like the main heating system).

2.3.9.2 The Override Hierarchy

Remember: *Command-line arguments are king.* If you have a database host set via `gsettings` but specify a different rule path or service-match via the terminal, the command-line flag will always take precedence. This allows you to experiment with "sandbox" configurations without breaking your main production setup.

3 System Architecture: Under the Hood

Now that you've mastered the language and the installation, it's time to look at the "nervous system" of STARTER. Understanding how the Core communicates with its services is key to building a truly robust automation network.

3.1 A Truly Distributed Ecosystem

As we touched upon in the introduction, STARTER is a *distributed system*. It isn't tied to a single CPU or a single box.

Single-Machine Mode
: For a home setup, one machine can handle everything.

Large-Scale Deployment
: In a smart city or industrial scenario, you can spread the workload across dozens of low-power machines.

The architecture follows a *Client/Server principle*. Each service that manages I/O acts as a *Server*, while the Core acts as the *Client*. You aren't limited to one client; you can deploy multiple Cores across a network, each handling a specific subset of services and logic.

3.1.1 ZeroConf: Plug & Play Reality

One of the most frustrating parts of automation is manual network configuration. STARTER eliminates this via *ZeroConf*.

The Core constantly listens for *mDNS (Multicast DNS)* events. The moment a new service is powered on and publishes itself on the network, the Core recognizes it, initiates a connection, and synchronizes automatically. No IP addresses to type, no config files to update.

3.1.2 Communication via D-Bus RPC

The "dialogue" between clients and servers happens over the *D-Bus TCP protocol*. This Remote Procedure Call (RPC) mechanism allows for high-speed, reliable data exchange.

When a service is discovered, the Core performs a "handshake" using three primary methods. Importantly, it always takes a full *snapshot* first and only *then* subscribes to change signals—so no early updates are lost in the gap between the two steps.

`getInputs`
: The Core requests a list of all available inputs from the service. After the snapshot is complete, it "hooks" into specific `*-changed` *signals*. This ensures that the moment a sensor value shifts at the service level, the Core is notified instantly.

`getOutputs`
: Similar to inputs, the Core fetches the array of available outputs. It hooks the change signals here as well, keeping the Core's internal map perfectly synchronized with the service's real-world state.

`getStats` Finally, the Core retrieves the service's internal statistics. Like the I/O, these are monitored via signals so that any diagnostic or performance data is updated in real-time.

This handshake can run synchronously or asynchronously. The Core prefers the asynchronous path so that discovering many peers at once does not stall the main loop.

3.1.3 The Operational Loop

Once the handshake is complete, the Client and Server are fully linked. Here is the lifecycle of a single action:

Change A service updates an Input (e.g., a thermometer reads a new temperature).

Notification The service fires a signal; the Core receives it immediately.

Processing The Core re-evaluates all rules related to that Input.

Action If a rule triggers a change in an Output, the Core calls the `SetOutput` method on the service.

Alignment The service receives the command and adjusts the physical or abstract hardware to match the requested value.

3.2 The Debug Server

When you need to observe or steer a live Core without installing a second `STClient` against every peer, build with `--enable-debug-server` and start the process with `--debug-server-bind=`*`HOST`*`:`*`PORT`* (default `0.0.0.0:17373`). The Core then opens a dedicated *GDBus over TCP* endpoint that external tools—such as a future debugger CLI—can attach to.

This channel is intentionally separate from the service bus: the debugger talks only to the Core. Peer I/O still flows through the Core's existing clients; the debug interface never opens a competing connection to a service.

3.2.1 Connecting to the Debug Endpoint

Clients connect with a GDBus address of the form `tcp:host=`*`HOST`*`,port=`*`PORT`*. Anonymous authentication is allowed. Once connected, register on:

`Object path`
: `/org/starter/core/debug`

`Interface`
: `org.starter.Core.Debug`

The exact client address string is also reported by `GetStatus` under the `client-address` key (useful when the bind used port `0` and the kernel picked an ephemeral port).

3.2.2 Debug Methods

`GetStatus () ⇒ (a{sv})`
: Snapshot of process state. Dictionary keys include `package`, `version`, `rules-file`, `service-match`, `bind-host`, `bind-port` (`q`), `client-address`, `peers` / `pending-peers` / `inputs` / `outputs` (`u`), `no-discovery` (`b`), `paused` (`b`), `stop-reason` / `stop-id` (`s`), and counts of `breakpoints` / `watchpoints` (`u`).

`ListPeers () ⇒ (a(ssuu))`
: Current peer table. Each tuple is (*name*, *address*, *port*, *state*), where *state* matches `STClientState` (`IDLE`=0, `PENDING`=1, `READY`=2, `FAILED`=3).

`ListInputs () ⇒ (a{s(siddddt)})`
`ListOutputs () ⇒ (a{s(siddddt)})`
: Full snapshot of the Core's local I/O tables (`all_inputs` / `all_outputs`). Each dict entry uses the same shape as `InputChanged` / `OutputChanged`: key is the identifier; value is (*description*, *unit*, *min*, *max*, *step*, *value*, *flags*). Debuggers should call these on attach and for `info io` so stable values (never recently changed) are still visible.

`SetInput (s id, d value, t flags) ⇒ (i ok)`
: Writes into the Core's local input cache for *id*, as if the peer had reported a new value. This triggers the normal `val-changed` path (and therefore rules), unless a breakpoint or watchpoint pauses evaluation first. Returns `1` on success, `0` on failure.

`SetOutput (s id, d value, t flags) ⇒ (i ok)`
: Calls `st_client_set_output` on the owning peer, bypassing `MANUAL_OVERRIDE` gating so a debugger can force actuators during a session. Returns `1` on success, `0` on failure.

`AddBreakpoint (s id) ⇒ (u handle)`
`RemoveBreakpoint (u handle) ⇒ (b ok)`
`ListBreakpoints () ⇒ (a(us))`
: Breakpoints are keyed by I/O or variable identifier. A hit occurs when rules are about to run because of that *id* (value change *or* apply). The Core then pauses before `engine_apply`. List entries are (*handle*, *id*).

`AddWatchpoint (s id) ⇒ (u handle)`
`RemoveWatchpoint (u handle) ⇒ (b ok)`
`ListWatchpoints () ⇒ (a(us))`
: Watchpoints fire only when the value of *id* changes (input, output, or variable). Same pause semantics as breakpoints.

`Continue ()`
: Clears the paused state so rule evaluation may proceed again.

`GetPaused () ⇒ (b paused)`
: `TRUE` while the Core is cooperatively stopped after a breakpoint or watchpoint hit.

3.2.3 Debug Signals

While connected, the Core broadcasts live events on the same interface:

`PeerChanged (s name, s address, u port, u state, s event)`
: Peer lifecycle. *event* is one of `connecting`, `ready`, `failed`, or `removed`. On `removed`/`failed` the peer is dropped from the `ListPeers` table.

`InputChanged ({s(siddddt)} object)`
`OutputChanged ({s(siddddt)} object)`
: Same dict entry shape used by `org.starter.Service` snapshots: key is the identifier; value is `(description, unit, min, max, step, value, flags)`.

`StatsChanged (s name)`
: A peer's statistics object was updated (*name* identifies the stats bundle).

`VarChanged (s name, s describe)`
: A Core variable changed; *describe* is the expression engine's textual dump of the new value.

`Stopped (s reason, s id, d value, u handle)`
: Emitted when evaluation pauses. *reason* is `breakpoint` or `watchpoint`; *id* and *handle* identify the point that fired.

3.2.4 Pause Semantics

Pause is *cooperative*: history recording and sniff signals keep running, but the Core skips `engine_apply` until `Continue`. While already paused, further matching intercepts stay paused (they do not queue additional stops). A full re-evaluation with a `NULL` trigger id (for example after a peer becomes ready) honors an existing pause but does not newly match breakpoints by id.

Use breakpoints when you care that rules are about to run for an identifier; use watchpoints when you only care that its value moved.

4 The starter-core Library: Building Your Own Ecosystem

The heavy lifting—networking, D-Bus signaling, and service discovery—is already baked into the library. This means you can focus on your specific logic while the library handles the "plumbing."

Let's dive into the anatomy of a service. We'll build a template for your first service in just a few lines of C.

4.1 The Skeleton: `first-service.c`

```
#include <stdlib.h>
#include <glib.h>

#include <starter/mdns.h>
#include <starter/server.h>
#include <starter/input.h>
#include <starter/output.h>

static GPtrArray *inputs = NULL;
static GPtrArray *outputs = NULL;
static STVersion version = { 0, 1, 0 };

int main(int argc, char *argv[])
{
  GMainLoop *gloop = NULL;
  STServer *server;
  STMDNS *mdns;
  GError *error = NULL;

  gloop = g_main_loop_new(NULL, FALSE);

  // Initializing I/O containers
  inputs = g_ptr_array_new_full(0, g_object_unref);
  outputs = g_ptr_array_new_full(0, g_object_unref);

  // Creating the Server instance
  server = st_server_new("ST:MyFirstService", &version, inputs, outputs);
  st_server_start(server, 0, &error);

  // Publishing the service via ZeroConf (mDNS)
  mdns = st_mdns_new();
  st_mdns_publish_service(mdns, st_server_get_name(server), "_dbus._tcp", st_server_ge

  g_main_loop_run(gloop);

  return EXIT_SUCCESS;
}
```

4.1.1 Compiling and Running

To turn this code into a binary, use `gcc` along with `pkg-config` to automatically pull in the necessary flags and libraries:

```
gcc -o my-first-service my-first-service.c `pkg-config --cflags --libs starter-core`
```

When you run it, it might look like it's doing nothing. But by enabling debug logs, you can watch the handshake happen in real-time:

```
export G_MESSAGES_DEBUG=libstarter-core
./my-first-service
```

If the Core is already running, you'll see your service "answering" the Core's queries:

```
... handle method call GetInputs
... handle method call GetOutputs
... handle method call GetStats
```

4.1.2 Breaking Down the Code

`st_server_new(...)`
: This initializes your service. You provide a unique name (like `ST:MyFirstService`), a version, and the I/O arrays. A key feature here is *shared memory*: the library doesn't copy your inputs and outputs; it shares them. If your code updates a sensor value, the library knows immediately. If the Core sends a command to an output, your code is notified.

`st_server_start(server, 0, ...)`
: This starts the D-Bus server on a local port. By passing `0`, the library automatically finds the first available port—no more manual port management.

`st_mdns_new()`
`st_mdns_publish_service(...)`
: This is where the "magic" happens. The first call sets up the discovery engine; the second broadcasts your service's name and port to the network. This is the broadcast that the Core hears, triggering the automatic connection.

4.2 Adding Substance: Inputs and Outputs

In the STARTER ecosystem, a service exists to share data. To do that, we need to populate our *inputs* and *outputs* arrays. Let's upgrade our code to version *0.2.0*:

```
// ... headers as before ...

int main(int argc, char *argv[])
{
  // ... boilerplate initialization ...

  // Defining a new Input
  inputs = g_ptr_array_new_full(0, g_object_unref);
  g_ptr_array_add(inputs, st_input_new("myFirstInput",
    "My first input", ST_UNIT_NULL, 0, 1, 1, 0, ST_IO_FLAG_NULL));
```

```
  // Defining a new Output
  outputs = g_ptr_array_new_full(0, g_object_unref);
  g_ptr_array_add(outputs, st_output_new("myFirstOutput",
    "My first output", ST_UNIT_NULL, 0, 1, 1, 0, ST_IO_FLAG_NULL));

  // ... server creation and mDNS publish ...
}
```

4.2.1 The Anatomy of an I/O Object

To create these objects, we use the `st_input_new` and `st_output_new` functions. Their signatures are identical, but their roles in the system logic are opposite.

```
STInput * st_input_new(
    const gchar *name,         // Unique identifier (e.g., "temperature")
    const gchar *description,  // Human-readable description
    STUnit unit,               // Unit of measurement (Celsius, Percentage, etc.)
    gdouble min,               // Minimum allowed value
    gdouble max,               // Maximum allowed value
    gdouble step,              // Resolution/Step of the value
    gdouble val,               // Initial value
    guint64 flags              // Special behavior flags
);
```

4.2.2 The Power of Flags: Customizing I/O Behavior

A critical, yet often overlooked, field in both `STInput` and `STOutput` objects is the *flags* field. These bitwise flags allow you to modify how the Core interacts with a specific data point, giving you the power to bypass standard logic or optimize database performance on the fly.

The system defines the following flags:

`ST_IO_FLAG_NULL`
: No flags set. The I/O behaves according to standard system defaults.

`ST_IO_FLAG_MANUAL_OVERRIDE`
: This is a "safety" flag for Outputs. When set, the Core is prohibited from automatically changing the value. This is essential for maintenance modes or when you need to take manual control of an actuator without the logic blocks interfering.

`ST_IO_FLAG_NO_HISTORY`
: Use this to keep your database lean. If an Input or Output has this flag, the Core will process the data in real-time but will not write it to the history tables. Perfect for high-frequency data that doesn't require long-term logging.

`ST_IO_FLAG_USR1`
`ST_IO_FLAG_USR2`
`ST_IO_FLAG_USR3`
`ST_IO_FLAG_USR4`
: These are reserved for you. They carry no internal meaning for the STARTER Core, allowing you to implement custom logic filters or tagging within your own services.

By mastering these flags, you aren't just defining data; you are defining the policy of that data within your distributed network.

4.3 Signals: The Reactive Core

These objects are derived from `GObject`, which means they are "chatty." They communicate changes through signals. Every time you modify a property, the object alerts the system.

`changed` The "master" signal. It fires whenever *any* property of the object changes.

`name-changed`
`descr-changed`
`unit-changed`
`min-changed`
`max-changed`
`step-changed`
`val-changed`
`flags-changed`
: These fire only when their specific property is updated.

For instance, if you update the *min* value of a sensor, both `changed` and `min-changed` will be emitted. This allows the Core (or other parts of your code) to react only to the information that matters.

4.3.1 Input vs. Output: What's the difference?

If the parameters are identical, why do we have two different types?

`STInput` Represents data coming *from the service to the Core* (e.g., a sensor reading). The service "owns" the value; the Core consumes it.

`STOutput` Represents data coming *from the Core to the service* (e.g., a command to turn on a light). The Core "requests" a value; the service executes it.

4.4 The Grand Finale: Bridging Code and Logic

An *Input* is a stream of data to be read; an *Output* is a command waiting to be written. Now that we've defined our I/O, let's stop talking and start *acting*.

We're going to evolve our service into a living entity. We'll implement a *sensor simulation* (a timer that increments a value) and a *callback* that waits for the Core to bark an order.

4.4.1 The Living Service: `first-service-v3.c`

```
#include <starter/mdns.h>
```

```
#include <starter/server.h>
#include <starter/input.h>
#include <starter/output.h>

static GPtrArray *inputs = NULL;
static GPtrArray *outputs = NULL;
static STVersion version = { 0, 3, 0 };

// Simulation: Incrementing our virtual sensor every second
static gboolean update_input (gpointer data) {
    STInput *input = ST_INPUT(data);
    gdouble val = st_input_get_val(input);

    val = (val < 100) ? val + 1 : 0;

    st_input_set_val(input, val, NULL); // This triggers the *-changed signal automati
    return G_SOURCE_CONTINUE;
}

// Callback: This fires the moment the Core sends a command to our service
static void on_output_changed(STOutput *out, gpointer data) {
    g_print("Action Triggered! Core set output to: %lf\n", st_output_get_val(out));
}

int main(int argc, char *argv[]) {
    GMainLoop *gloop = g_main_loop_new(NULL, FALSE);
    STServer *server;
    STMDNS *mdns;

    inputs = g_ptr_array_new_full(0, g_object_unref);
    g_ptr_array_add(inputs, st_input_new("myFirstInput", "Virtual Sensor", ST_UNIT_NUL

    outputs = g_ptr_array_new_full(0, g_object_unref);
    g_ptr_array_add(outputs, st_output_new("myFirstOutput", "Actuator", ST_UNIT_NULL,

    // CONNECTING THE SIGNAL: Tell the code to listen for "val-changed"
    g_signal_connect(outputs->pdata[0], "val-changed", G_CALLBACK(on_output_changed),

    server = st_server_new("ST:MyFirstService", &version, inputs, outputs);
    st_server_start(server, 0, NULL);
    mdns = st_mdns_new();
    st_mdns_publish_service(mdns, st_server_get_name(server), "_dbus._tcp", st_server_

    // Start the heartbeat: Update the input every 1 second
    g_timeout_add_seconds(1, update_input, inputs->pdata[0]);

    g_main_loop_run(gloop);
```

```
    return EXIT_SUCCESS;
}
```

4.4.2 The Rule: Closing the Loop

With the service broadcasting and listening, we just need the "brain" to make a decision. Open your rules file (see `rules.conf.example`) and drop this in:

```
myFirstOutput {
    SET 1 WHEN myFirstInput > 50;
    SET 0 WHEN TRUE;
}
```

4.4.3 The Result: A Symphony of Data

Here is what happens when you hit "Run":

Discovery Your service shouts its existence over mDNS; the Core hears it and hooks up via D-Bus.

Telemetry Your code starts counting. Every second, the Core receives the updated `myFirstInput`.

Threshold The moment that counter hits 51, the Core's rule evaluates to `TRUE`.

Reaction The Core calls `SetOutput` on your service. Your `on_output_changed` function fires, and you see the confirmation in your console.

Congratulations You've just built a fully distributed, reactive automation loop. You've bridged the gap between raw C code and high-level abstract logic. This is no longer just a "beep example"—it's the blueprint for an entire ecosystem.

5 Conclusions: The World is Your Input

We've covered the syntax, the distributed architecture, and the "plumbing" that makes STARTER tick. But as we reach the end of this journey, the most important tool I can leave you with isn't a library or a command—it's *the art of abstraction.*

5.1 Thinking in STARTER

Building a smart system isn't about connecting wires; it's about translating reality into a digital dialogue. When you look at your home, your office, or a complex industrial plant, stop seeing "machines" and start seeing *Inputs* and *Outputs.*

- *Is it a sensor?* It's an Input. Whether it's a thermometer, a stock market ticker, or a database query monitoring server load—it's just a number flowing into the system.
- *Is it an action?* It's an Output. Turning on a heater, sending an alert to your phone, or scaling a cloud instance—it's just a value being set by the Core.

5.1.1 The Power of Generalization

The beauty of the STARTER architecture is that the Core doesn't care *what* the data represents. To the rules engine, a temperature of 25 °C and an oil price of $25 are identical. This is your greatest advantage. By generalizing your data, you can reuse logic, simplify complex scenarios, and create an ecosystem where unrelated services can finally "talk" to each other through the Core.

5.1.2 Where Will You Take It?

STARTER was born from the need for a system that is as simple as a light switch but as powerful as a distributed cluster. We've given you the engine; now you are the architect.

Whether you're automating a single room to react to the sun or orchestrating a network of machines across different continents, the principle remains the same: *Abstract the complexity, define your rules, and let the system handle the rest.*

Now, the keyboard is yours. What will be your first service? What rules will you write to make your environment work for you?

Go ahead. Start something.

Appendix A Database Schema & Initialization

As we explored in the previous chapters, STARTER is a data-driven ecosystem. For the Core to manage persistence, track event history, and monitor system performance, it requires a solid and correctly configured storage infrastructure.

This appendix provides the SQL queries necessary to initialize the MariaDB/MySQL database, which should be named `STARTER`. The schema is built around three foundational tables:

`labels` Maps I/O identifiers to human-readable descriptions and metadata.

`history` The primary ledger where every single event and state change is recorded for long-term analysis.

`stats` Dedicated to gathering diagnostic data and runtime statistics from your various services.

```
CREATE DATABASE STARTER;

USE STARTER;

CREATE TABLE labels (
    id INT NOT NULL AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT NOT NULL,
    type VARCHAR(50) NOT NULL,
    unit INT NOT NULL,
    min DOUBLE NOT NULL,
    max DOUBLE NOT NULL,
    step DOUBLE NOT NULL,
    PRIMARY KEY (id),
    INDEX idx_labels_type (type)
);

CREATE TABLE history (
    id BIGINT NOT NULL AUTO_INCREMENT,
    timestamp DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    source VARCHAR(100) NOT NULL,
    label_id INT NOT NULL,
    value DOUBLE NOT NULL,
    flags BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (label_id) REFERENCES labels(id),
    INDEX idx_label_time (label_id, timestamp),
    INDEX idx_history_source (source)
);

CREATE TABLE stats (
    id                INTEGER PRIMARY KEY AUTO_INCREMENT,
```

```
    service_name      VARCHAR(100) NOT NULL UNIQUE,
    updated_at        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    vmsize_min        REAL NOT NULL,
    vmsize_max        REAL NOT NULL,
    vmsize_last       REAL NOT NULL,

    vmrss_min         REAL NOT NULL,
    vmrss_max         REAL NOT NULL,
    vmrss_last        REAL NOT NULL,

    vmswap_min        REAL NOT NULL,
    vmswap_max        REAL NOT NULL,
    vmswap_last       REAL NOT NULL,

    io_read_min       REAL NOT NULL,
    io_read_max       REAL NOT NULL,
    io_read_last      REAL NOT NULL,

    io_write_min      REAL NOT NULL,
    io_write_max      REAL NOT NULL,
    io_write_last     REAL NOT NULL,

    cpu_min           REAL NOT NULL,
    cpu_max           REAL NOT NULL,
    cpu_last          REAL NOT NULL,

    icswitch_min      REAL NOT NULL,
    icswitch_max      REAL NOT NULL,
    icswitch_last     REAL NOT NULL,

    priority_min      REAL NOT NULL,
    priority_max      REAL NOT NULL,
    priority_last     REAL NOT NULL,

    threads_min       REAL NOT NULL,
    threads_max       REAL NOT NULL,
    threads_last      REAL NOT NULL,
    INDEX idx_service_name (service_name)
);
```

Security Note: Ensure the *db-user* defined in your `gsettings` configuration has full Read/Write permissions (`GRANT ALL PRIVILEGES`) on these tables. Without this, the Core will be unable to commit the "memory" of your system to the disk.

Appendix B API Reference

B.1 Error Handling: The `STError` Domain

In a complex ecosystem where services communicate over the network via D-Bus, error handling is not just an option—it is a necessity. STARTER uses the GLib `GError` system to report failures in a standardized way.

B.1.1 STError Enumeration

The `STError` enumeration defines the specific error codes returned by the library. These errors are associated with the `ST_ERROR` quark.

`STError` [Enumeration]

This enumeration defines the failure modes that can occur during I/O operations or service communication.

`ST_ERROR_INVALID_VALUE`
: The value provided for an operation is outside the allowed range (defined by *min* and *max*) after snapping to *step*, or is otherwise incompatible with the expected constraints. Off-step values are rounded rather than rejected.

`ST_ERROR_INVALID_RESPONSE`
: The remote service or the Core sent a response that does not match the expected D-Bus signature or contains malformed data.

`ST_ERROR_NOT_IMPLEMENTED`
: The requested method or operation is recognized but not currently supported by the specific version of the service or the library.

`ST_ERROR_NOT_CONNECTED`
: The client is not connected (or not yet `READY`) and cannot perform the requested remote operation.

`ST_ERROR_SYSTEM`
: A lower-level system or library failure occurred (for example, an unexpected GLib or network error wrapped into the STARTER domain).

B.1.2 Error Quarks and Macros

To work with GLib's error reporting system, STARTER defines a specific error domain.

`ST_ERROR` [Macro]

This macro expands to the result of `st_error_quark()`. It is used to identify the error domain when catching or throwing errors:

```
if (error->domain == ST_ERROR) {
    /* Handle STARTER specific errors */
}
```

`GQuark st_error_quark` (*void*) [Function]

Returns a `GQuark` that uniquely identifies the STARTER error domain. This function is typically called via the `ST_ERROR` macro.

B.2 API Definitions: Constants and Enumerations

This section describes the foundational constants and types defined in `starter/defs.h`. These definitions are shared across the entire ecosystem to ensure consistent behavior between services and the Core.

B.2.1 I/O Flags

Flags are used to modify the behavior of `STInput` and `STOutput` objects. They are defined as bitwise constants of type `guint64`.

`ST_IO_FLAG_NULL`
: No flags set (`0x0`). Standard system behavior applies.

`ST_IO_FLAG_MANUAL_OVERRIDE`
: When set on an Output, it prevents the Core from automatically updating its value. This is used for manual maintenance or safety locks.

`ST_IO_FLAG_NO_HISTORY`
: Disables database logging for the specific I/O. The Core processes changes in real-time but does not record them in the `history` table.

`ST_IO_FLAG_USR1`
`ST_IO_FLAG_USR2`
`ST_IO_FLAG_USR3`
`ST_IO_FLAG_USR4`
: User-defined flags. These have no internal meaning for the Core and can be used by developers for custom service-level logic.

`ST_IO_FLAG_USR_MASK`
: A bitmask (`0xF000000000000000LL`) provided to easily isolate or clear all user-defined flags.

B.2.2 STUnit Enumeration

The `STUnit` enumeration defines the measurement units supported by the system. Every Input and Output must be associated with a unit to ensure proper data interpretation and visualization (e.g., in Grafana dashboards).

`STUnit` [Enumeration]

`ST_UNIT_NULL`
: No unit specified.

`ST_UNIT_CELSIUS`
: Temperature in degrees Celsius.

`ST_UNIT_PERCENT`
: Percentage values (0.0 to 100.0).

`ST_UNIT_BAR`
: Pressure in bars.

`ST_UNIT_ONOFF`
: Binary state representation.

`ST_UNIT_ENUM`
: A discrete list of states.

`ST_UNIT_V`
`ST_UNIT_W`
`ST_UNIT_VAR`
`ST_UNIT_KW`
: Electrical units: Volts, Watts, Volt-Ampere Reactive, and Kilowatts.

`ST_UNIT_WH`
`ST_UNIT_KWH`
: Energy consumption units: Watt-hours and Kilowatt-hours.

`ST_UNIT_SECONDS`
`ST_UNIT_MINUTES`
`ST_UNIT_HOURS`
: Time duration units.

B.2.3 Special Constants

`ST_INVALID_VALUE` [Macro]
: This macro evaluates to `G_MAXDOUBLE`. It is used by the system to indicate that a value is currently unavailable or that an I/O has not yet been initialized.

B.3 STInput: The Sensor Abstraction

The `STInput` object represents a single data source within the STARTER ecosystem. It is a derivable `GObject` that encapsulates not only the current value but also the metadata (units, ranges, steps) required for the Core to interpret the data correctly.

B.3.1 Lifecycle Management

`STInput * st_input_new` (*const gchar *`name`*, *const gchar *`description`*, *STUnit* `unit`, *gdouble* `min`, *gdouble* `max`, *gdouble* `step`, *gdouble* `val`, *guint64* `flags`) [Function]
: Creates a new `STInput` instance.

- *name*: Unique identifier used in rules (e.g., "living_room_temp").
- *description*: A human-readable string describing the input.
- *unit*: The `STUnit` associated with the data.
- *min* / *max*: The allowed logical range for the value.
- *step*: The minimum increment between values.
- *val*: The initial value.
- *flags*: Behavior modifiers (see Section B.2.1 [I/O Flags], page 31).

`STInput * st_input_dup` (*const STInput *`self`*) [Function]
: Creates a deep copy of an existing `STInput` object.

B.3.2 Metadata Accessors

These functions manage the identity and properties of the input. Most "set" functions will emit a specific signal (e.g., `name-changed`) and a general `changed` signal upon success.

`const char * st_input_get_name` (*const STInput *`self`*) [Function]
`gboolean st_input_set_name` (*STInput *`self`*, *const char *`name`*, *GError **`error`*) [Function]
Gets or sets the unique identifier for the input.

`const char * st_input_get_description` (*const STInput *`self`*) [Function]
`gboolean st_input_set_description` (*STInput *`self`*, *const char *`description`*, *GError **`error`*) [Function]
Gets or sets the human-readable description.

`STUnit st_input_get_unit` (*const STInput *`self`*) [Function]
`gboolean st_input_set_unit` (*STInput *`self`*, *STUnit `unit`*, *GError **`error`*) [Function]
Gets or sets the measurement unit.

B.3.3 Range and Value Control

`gdouble st_input_get_val` (*const STInput *`self`*) [Function]
`gboolean st_input_set_val` (*STInput *`self`*, *gdouble `val`*, *GError **`error`*) [Function]
Returns or updates the current value. Values that are not aligned with *step* are rounded to the nearest multiple (a warning is logged when this happens). If the rounded value is outside the *min*/*max* range, the function fails with `ST_ERROR_INVALID_VALUE`.

`gdouble st_input_get_min` (*const STInput *`self`*) [Function]
`gboolean st_input_set_min` (*STInput *`self`*, *gdouble `min`*, *GError **`error`*) [Function]
`gdouble st_input_get_max` (*const STInput *`self`*) [Function]
`gboolean st_input_set_max` (*STInput *`self`*, *gdouble `max`*, *GError **`error`*) [Function]
Manage the operational boundaries of the input. Bounds are snapped to *step* the same way as values.

`gdouble st_input_get_step` (*const STInput *`self`*) [Function]
`gboolean st_input_set_step` (*STInput *`self`*, *gdouble `step`*, *GError **`error`*) [Function]
Manage the value increment resolution. Changing the step rounds the current *min*, *max*, and *val* onto the new grid (with warnings when they change). The call fails only if those rounded values are no longer ordered.

`guint64 st_input_get_flags` (*const STInput *`self`*) [Function]
`gboolean st_input_set_flags` (*STInput *`self`*, *guint64 `flags`*, *GError **`error`*) [Function]
Gets or sets the behavioral flags (e.g., `ST_IO_FLAG_NO_HISTORY`).

B.4 STOutput: The Actuator Abstraction

The `STOutput` object represents an actionable endpoint within the STARTER ecosystem. While `STInput` is for reading the world, `STOutput` is for changing it. It is a derivable `GObject` that filters incoming commands from the Core against defined ranges and steps.

B.4.1 Lifecycle Management

`STOutput * st_output_new` (*const gchar *`name`, const gchar *`description`, STUnit `unit`, gdouble `min`, gdouble `max`, gdouble `step`, gdouble `val`, guint64 `flags`*) [Function]

Creates a new `STOutput` instance.

- *name*: Unique identifier used as the target in rules (e.g., `main_heater`).
- *description*: Human-readable description of the actuator.
- *unit*: The `STUnit` (e.g., `ST_UNIT_ONOFF` for switches).
- *min* / *max*: The allowed range for commanded values.
- *step*: The resolution for valid commands.
- *val*: The initial state of the output.
- *flags*: Behavior modifiers (see Section B.2.1 [I/O Flags], page 31).

`STOutput * st_output_dup` (*const STOutput *`self`*) [Function]

Creates a deep copy of an existing `STOutput` object.

B.4.2 Metadata Accessors

Similar to inputs, these functions manage the properties of the output. Setting these properties will emit specific signals to notify the rest of the system of the change in configuration.

`const char * st_output_get_name` (*const STOutput *`self`*) [Function]
`gboolean st_output_set_name` (*STOutput *`self`, const char *`name`, GError **`error`*) [Function]

Gets or sets the unique identifier.

`const char * st_output_get_description` (*const STOutput *`self`*) [Function]
`gboolean st_output_set_description` (*STOutput *`self`, const char *`description`, GError **`error`*) [Function]

Gets or sets the human-readable description.

`STUnit st_output_get_unit` (*const STOutput *`self`*) [Function]
`gboolean st_output_set_unit` (*STOutput *`self`, STUnit `unit`, GError **`error`*) [Function]

Gets or sets the measurement unit.

B.4.3 Value and State Control

`gdouble st_output_get_val` (*const STOutput *`self`*) [Function]

`gboolean st_output_set_val (STOutput *self, gdouble val, GError **error)` [Function]

Returns or updates the current value. For outputs, `st_output_set_val` is typically called by the service when it receives a command from the Core, or when a manual override is in effect. Values that are not aligned with *step* are rounded to the nearest multiple (a warning is logged when this happens). Values that remain outside *min*/*max* after rounding are rejected with `ST_ERROR_INVALID_VALUE`.

`gdouble st_output_get_min (const STOutput *self)` [Function]
`gboolean st_output_set_min (STOutput *self, gdouble min, GError **error)` [Function]
`gdouble st_output_get_max (const STOutput *self)` [Function]
`gboolean st_output_set_max (STOutput *self, gdouble max, GError **error)` [Function]

Manage the operational range. Bounds are snapped to *step*; commands that remain outside the range after rounding are rejected.

`gdouble st_output_get_step (const STOutput *self)` [Function]
`gboolean st_output_set_step (STOutput *self, gdouble step, GError **error)` [Function]

Sets the precision required for value updates. Changing the step rounds the current *min*, *max*, and *val* onto the new grid (with warnings when they change).

`guint64 st_output_get_flags (const STOutput *self)` [Function]
`gboolean st_output_set_flags (STOutput *self, guint64 flags, GError **error)` [Function]

Manages behavioral flags like `ST_IO_FLAG_MANUAL_OVERRIDE`.

B.5 Network Discovery: mDNS and Service Location

STARTER uses Multicast DNS (mDNS) to achieve true zero-configuration networking. This allows the Core and various services to discover each other dynamically on the local network.

B.5.1 STMDNSService: Service Representation

The `STMDNSService` object is a data container that represents a discovered service or a service ready to be published.

`STMDNSService * st_mdns_service_new (const gchar *name, const gchar *type, const gchar *host, const gchar *address, const guint16 port, const gint proto, const gint iface)` [Function]

Creates a new `STMDNSService` descriptor.

- *name*: The service instance name (e.g., `"ST:MyFirstService"`).
- *type*: The mDNS service type (e.g., `"_dbus._tcp"`).
- *host*: The target hostname.
- *address*: The IP address of the service.
- *port*: The TCP/UDP port where the service is listening.
- *proto*: The network protocol (IPv4/IPv6).

B.5.1.1 Property Management

`const gchar * st_mdns_service_get_name` (*const STMDNSService *`self`*) [Function]

`void st_mdns_service_set_name` (*STMDNSService *`self`*, *const gchar *`name`*) [Function]

`const gchar * st_mdns_service_get_mdns_type` (*const STMDNSService *`self`*) [Function]

`void st_mdns_service_set_mdns_type` (*STMDNSService *`self`*, *const gchar *`type`*) [Function]

`const gchar * st_mdns_service_get_host` (*const STMDNSService *`self`*) [Function]

`void st_mdns_service_set_host` (*STMDNSService *`self`*, *const gchar *`host`*) [Function]

`const gchar * st_mdns_service_get_address` (*const STMDNSService *`self`*) [Function]

`void st_mdns_service_set_address` (*STMDNSService *`self`*, *const gchar *`address`*) [Function]

`guint16 st_mdns_service_get_port` (*const STMDNSService *`self`*) [Function]

`void st_mdns_service_set_port` (*STMDNSService *`self`*, *const guint16 `port`*) [Function]

Getters and setters for the service network properties.

B.5.2 STMDNS: The Discovery Engine

The `STMDNS` object handles the actual background tasks of browsing for services and publishing your own.

`STMDNS * st_mdns_new` (*void*) [Function]

Initializes the mDNS engine.

`gboolean st_mdns_publish_service` (*STMDNS *`self`*, *const gchar *`name`*, *const gchar *`type`*, *guint16 `port`*) [Function]

Broadcasts your service to the network. This is what makes your service "visible" to the STARTER Core.

`const GList * st_mdns_get_all_services` (*STMDNS *`self`*) [Function]

Returns a list of all `STMDNSService` objects currently discovered on the network.

B.5.2.1 Signals

`STMDNS` *service-added* (`STMDNSService *service`) [Signal]

Emitted when a new STARTER-compatible service is discovered on the network.

`STMDNS` *service-removed* (`STMDNSService *service`) [Signal]

Emitted when a service is no longer reachable.

B.6 System Statistics: Performance Monitoring

STARTER provides a built-in framework for monitoring the health and performance of its services. This telemetry data allows for deep analysis of resource consumption, such as CPU usage and memory footprint.

B.6.1 STStats: The Telemetry Container

The `STStats` object acts as a snapshot of a process's performance metrics. It uses the `STStatsEntry` structure to store *min*, *max*, and *last* values for each metric.

`STStatsEntry` [Structure]

- `min` The minimum value recorded since the collector started.
- `max` The maximum value recorded.
- `last` The most recent measurement.

`STStats * st_stats_new` (*const gchar* `*name`) [Function]
Creates a new statistics container identified by *name*.

B.6.1.1 Metric Accessors

`const STStatsEntry * st_stats_get_cpu` (*const STStats* `*self`) [Function]
`const STStatsEntry * st_stats_get_vmrss` (*const STStats* `*self`) [Function]
`const STStatsEntry * st_stats_get_vmsize` (*const STStats* `*self`) [Function]
`const STStatsEntry * st_stats_get_threads` (*const STStats* `*self`) [Function]
Retrieve specific metrics including CPU load (percentage), Resident Set Size (memory actually used), Virtual Memory Size, and thread count.

B.6.2 STStatsCollector: The Abstract Engine

The `STStatsCollector` is a derivable base class designed to automate the sampling of statistics at regular intervals.

`STStatsCollector * st_stats_collector_new` (*STStats* `*stats`, *guint* `update_interval_ms`) [Function]
Initializes a collector that will update the provided *stats* object every *update_interval_ms* milliseconds.

B.6.3 STProcessStatsCollector: Process Monitoring

This is a specialized implementation of the collector that reads performance data directly from the system (e.g., `/proc` on Linux) for the current process.

`STProcessStatsCollector * st_process_stats_collector_new` (*STStats* `*stats`, *guint* `interval_ms`) [Function]
Creates a collector that automatically gathers OS-level metrics (memory, I/O, context switches) for the running service.

B.7 Services and Communication: Server and Client

This section details the primary objects used to establish distributed communication within the STARTER network. The `STServer` exposes local resources to the network, while `STClient` provides the means to discover and control remote services.

B.7.1 STServer: Publishing a Service

The `STServer` object is responsible for taking a collection of `STInput` and `STOutput` objects and making them available over D-Bus. It handles the low-level registration and message dispatching.

`STServer * st_server_new` (*const gchar *`name`, const STVersion *`version`, GPtrArray *`inputs`, GPtrArray *`outputs`*) [Function]

Initializes a new server instance.

- *name*: The unique name of the service (e.g., `"ST:MyService"`).
- *version*: The `STVersion` of the service.
- *inputs*: A `GPtrArray` containing the `STInput` objects to expose.
- *outputs*: A `GPtrArray` containing the `STOutput` objects to expose.

`gboolean st_server_start` (*STServer *`self`, guint16 `port`, GError **`error`*) [Function]

Starts the server and begins listening for incoming connections on the specified *port*. If *port* is `0`, the library asks the operating system for an ephemeral (dynamically allocated) port—no need to pick one by hand. After a successful start, call `st_server_get_port` to learn which port was actually bound; that value is what you advertise over mDNS and what a client must pass to `st_client_start`.

`guint16 st_server_get_port` (*const STServer *`self`*) [Function]

Returns the TCP port the server is listening on. Meaningful after a successful `st_server_start`; when the server was started with *port* `0`, this is the ephemeral port assigned by the system.

B.7.1.1 Service Metadata

`const gchar * st_server_get_name` (*const STServer *`self`*) [Function]

`gboolean st_server_set_name` (*STServer *`self`, const gchar *`name`, GError **`error`*) [Function]

`const STVersion * st_server_get_version` (*const STServer *`self`*) [Function]

`const GPtrArray * st_server_get_inputs` (*const STServer *`self`*) [Function]

`const GPtrArray * st_server_get_outputs` (*const STServer *`self`*) [Function]

Accessors to retrieve or modify the server's identity and its collection of I/O objects.

B.7.2 STClient: Consuming a Service

The `STClient` is the counterpart to the server. It connects to a remote `STServer` and creates a local proxy of all its inputs, outputs, and statistics.

Think of it as a living mirror: once the handshake completes, the Core can read remote sensors and command remote actuators as if they were local objects. The connection itself can be started *synchronously* (handy for tests and simple tools) or *asynchronously* (what the Core uses in production, so discovery never freezes the main loop).

In both cases the sequence is the same: take a full snapshot first (`GetInputs`, `GetOutputs`, `GetStats`), then subscribe to change signals. That ordering is deliberate—it closes the race where an early `*-changed` notification would otherwise arrive before the local mirrors exist.

`STClient * st_client_new` (*void*) [Function]
Creates a new client in the `ST_CLIENT_STATE_IDLE` state.

B.7.2.1 Client Connection State

A client is not a binary "connected / not connected" switch. It moves through an explicit state machine that tells you whether the mirrors are safe to use.

`STClientState` [Enumeration]

`ST_CLIENT_STATE_IDLE`
Freshly created; no connection attempt has started yet.

`ST_CLIENT_STATE_PENDING`
Connect and snapshot are in flight. Local I/O mirrors are not ready.

`ST_CLIENT_STATE_READY`
Snapshot complete, signals subscribed, mirrors usable.

`ST_CLIENT_STATE_FAILED`
The last start attempt failed (network error, cancelled task, invalid response, . . .).

`STClientState st_client_get_state` (*STClient *self*) [Function]
Returns the current connection state.

`gboolean st_client_is_ready` (*STClient *self*) [Function]
Convenience check equivalent to `st_client_get_state (self) == ST_CLIENT_STATE_READY`.

Calling `st_client_start` or `st_client_start_async` again while the state is already `PENDING` or `READY` is an error—create a new client, or wait until the current lifecycle ends.

B.7.2.2 Starting a Client

`gboolean st_client_start` (*STClient *self, const gchar *address, guint16 port, GError **error*) [Function]
Connects synchronously to a remote service at *address*:*port*, performs the initial snapshot, then subscribes to change signals. On success the client enters `ST_CLIENT_STATE_READY`; on failure it enters `ST_CLIENT_STATE_FAILED` and returns `FALSE`.

Unlike `st_server_start`, the client does *not* treat *port* `0` as "pick any port": it must be the real listening port of the peer (from mDNS, or from `st_server_get_port` when you own the server).

`void st_client_start_async` (*STClient* *`self`, *const gchar* *`address`, *guint16* `port`, *GCancellable* *`cancellable`, *GAsyncReadyCallback* `callback`, *gpointer* `user_data`) [Function]

Starts the same connect/snapshot/subscribe sequence without blocking the caller. Progress is reported through *callback*. Pass a `GCancellable` when you need to abort a pending start—for example when mDNS reports that the service disappeared before the handshake finished. As with the synchronous start, *port* must be the peer's actual listening port.

`gboolean st_client_start_finish` (*STClient* *`self`, *GAsyncResult* *`result`, *GError* **`error`) [Function]

Completes a previous `st_client_start_async` call. Returns `TRUE` when the client is `READY`.

B.7.2.3 Remote Resource Access

Once a client is `READY`, it provides access to the remote service's data as if it were local. Before that point the arrays may be empty or incomplete—always gate your logic on `st_client_is_ready`.

`const GPtrArray * st_client_get_inputs` (*STClient* *`self`) [Function]

`const GPtrArray * st_client_get_outputs` (*STClient* *`self`) [Function]

Returns arrays of `STInput` and `STOutput` objects that represent the remote service's resources.

`const STStats * st_client_get_stats` (*STClient* *`self`) [Function]

Retrieves the performance statistics (`STStats`) of the remote process.

`gchar * st_client_get_service_name` (*STClient* *`self`) [Function]

Returns the remote service name as a newly allocated string. Free it with `g_free` when you are done.

`gboolean st_client_set_output` (*STClient* *`self`, *const gchar* *`name`, *double* `value`, *guint64* `flags`) [Function]

Requests that the remote service set the named output. This call is still synchronous today; an asynchronous variant is planned so that rule evaluation never blocks the main loop on a slow peer.

Appendix C GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.
`https://fsf.org/`

Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document *free* in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or non-commercially. Secondarily, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released

under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

   If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

   If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

   If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

   It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

   You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

   A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any,

be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.

B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.

C. State on the Title page the name of the publisher of the Modified Version, as the publisher.

D. Preserve all the copyright notices of the Document.

E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.

F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.

G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.

H. Include an unaltered copy of this License.

I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.

J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.

K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.

L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.

M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.

N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.

O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their

titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

   You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

   The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

   In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements."

6. COLLECTIONS OF DOCUMENTS

   You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

   You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

   A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

   If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

   Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

   If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

   You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

   However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

   Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

   Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

    The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See `https://www.gnu.org/licenses/`.

    Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

    "Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration" (or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

    "CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

    "Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

    An MMC is "eligible for relicensing" if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

    The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

```
Copyright (C)  year  your name.
Permission is granted to copy, distribute and/or modify this document
under the terms of the GNU Free Documentation License, Version 1.3
or any later version published by the Free Software Foundation;
with no Invariant Sections, no Front-Cover Texts, and no Back-Cover
Texts.  A copy of the license is included in the section entitled ``GNU
Free Documentation License''.
```

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the "with. . . Texts." line with this:

```
with the Invariant Sections being list their titles, with
the Front-Cover Texts being list, and with the Back-Cover Texts
being list.
```

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.